Redundancy Does Not Imply Fault Tolerance:

Analysis of Distributed Storage Reactions to Single Errors and Corruptions

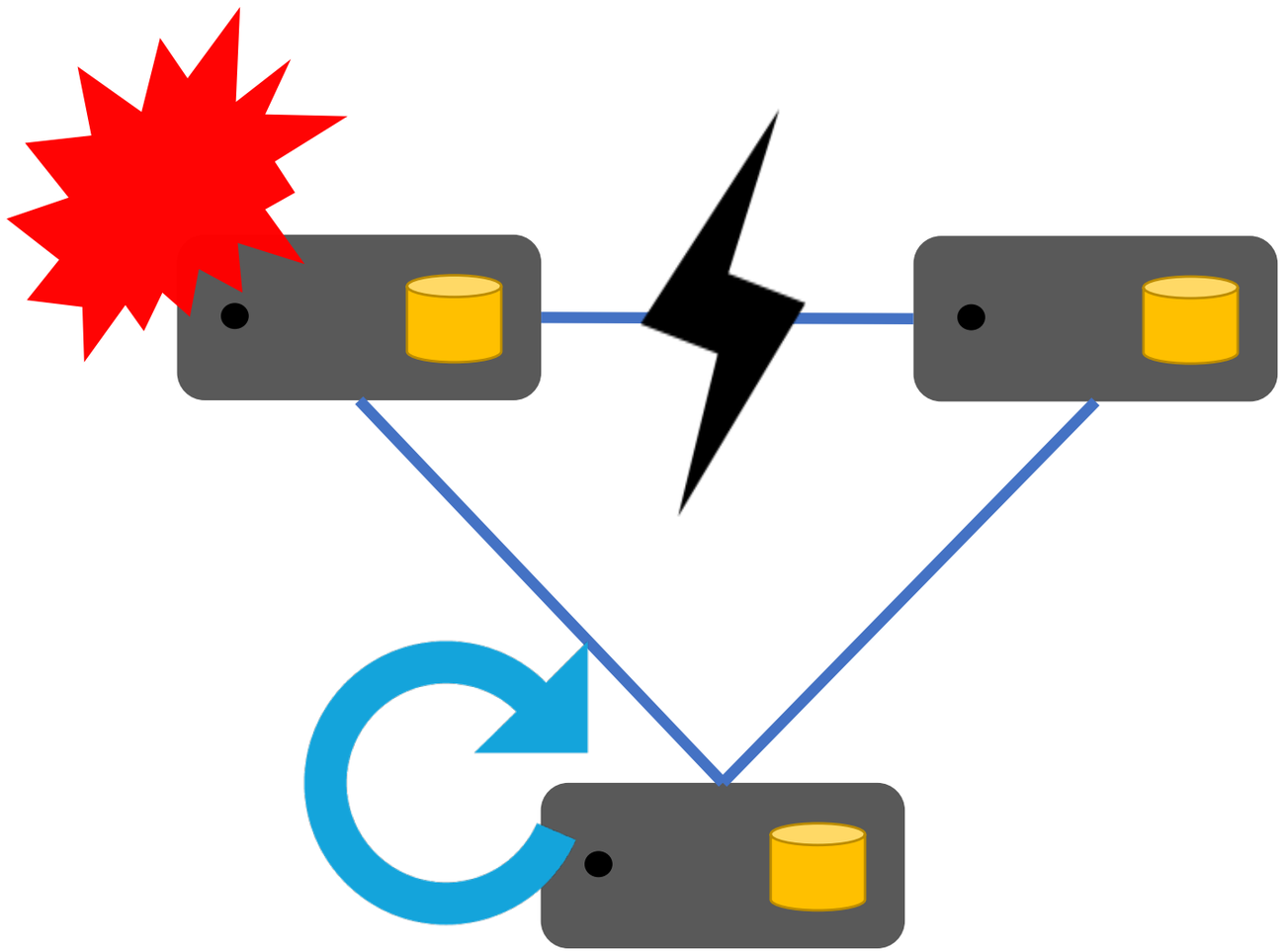
Aishwarya Ganesan, Ramnatthan Alagappan,
Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau

Why?
Analysis: Understand real systems (and problems) in detail
→ new solutions
→ generalize

Motivation?

Do distributed storage systems use *redundancy* to *recover* from local file-system faults?

Redundancy for Fault Tolerance



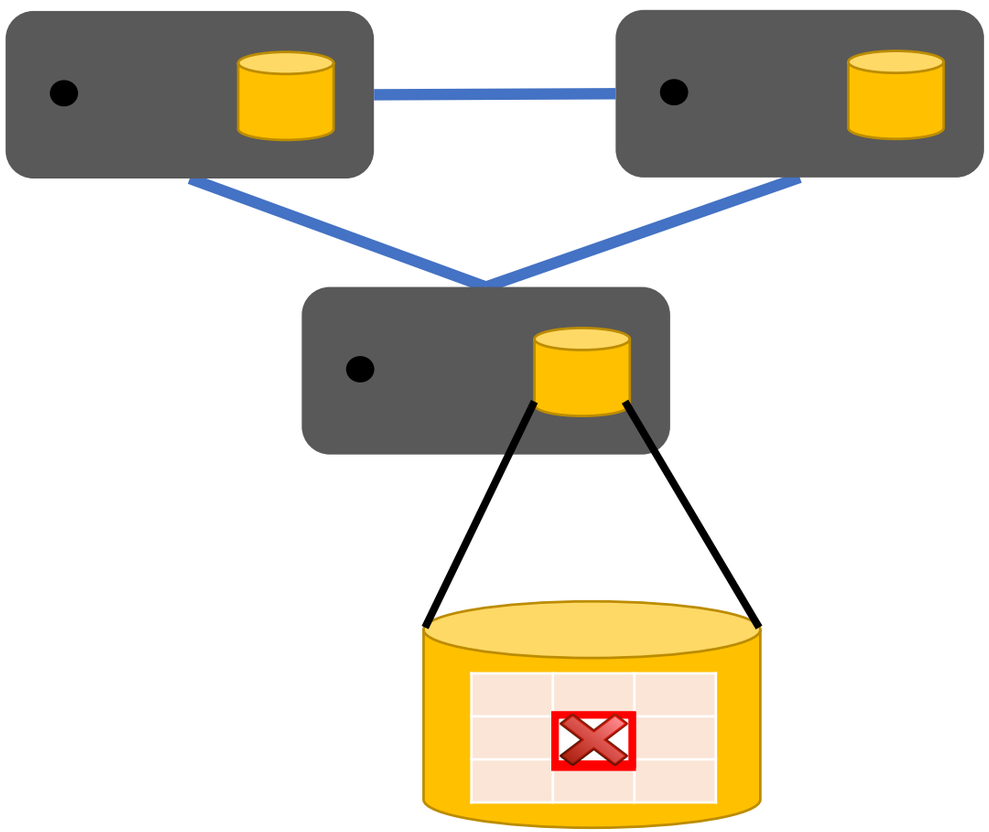
Replication can mask failures

- System crashes
- Machine reboots
- Network failures

What types of faults can occur?

Distributed service depends on local file systems to store data

How about partial storage faults?



Local FS may return corrupted data on reads

- disk block corruption

File system may return I/O error on read

- latent sector error

Refer to as **File-system faults**

Fault model

A single fault to a single file-system block in a single node

Type of faults:

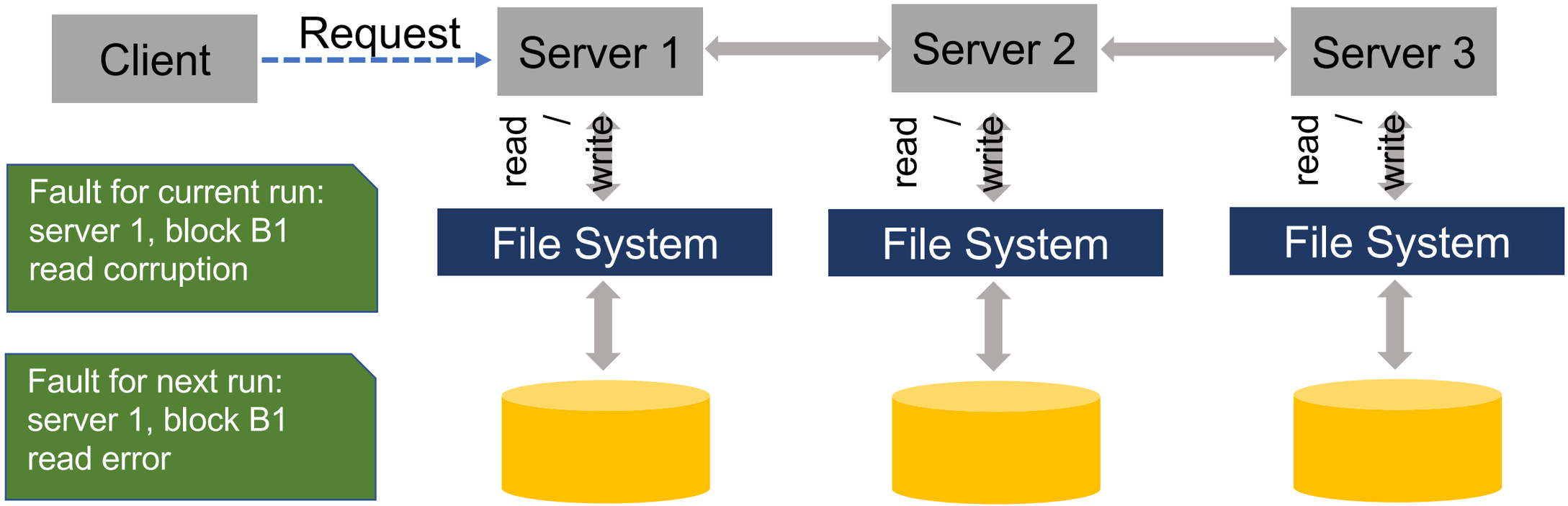
- corruptions rand data/0 data
- read errors errfs returns these errors nicely
- write errors

Fault model?

These are the questions being asked about the different softwares

- Are these faults fail-stop?
- Do the systems fail-fast?
- How do these faults happen in the real world?

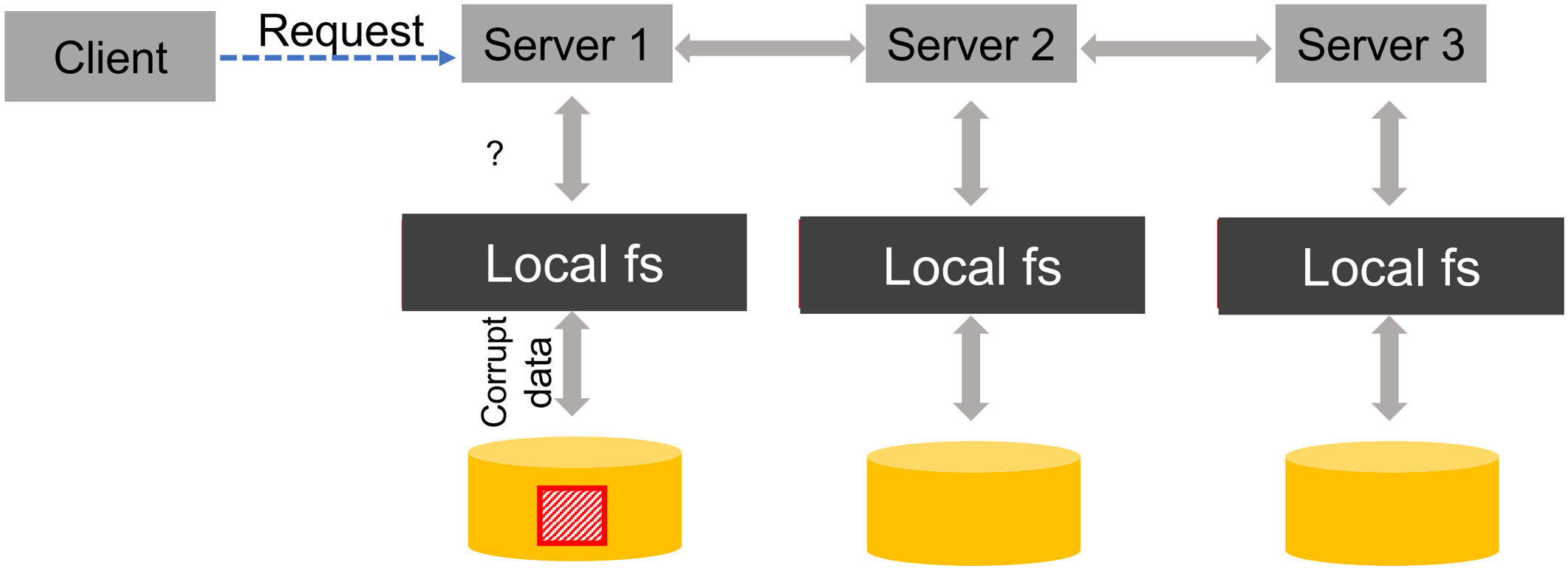
Fault Model



A single fault to a single file-system block in a single node

Faults injected only to user data not filesystem metadata

Fault Model: ext4 and btrfs

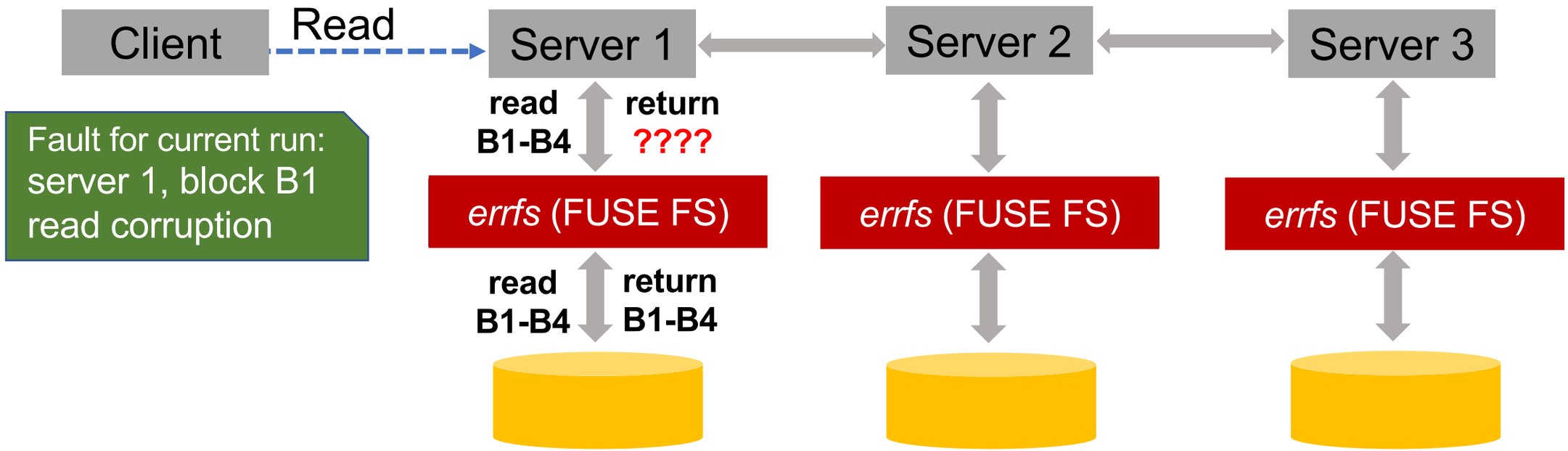


Ext4: disk corruption → corrupted data to applications

Btrfs: disk corruption → I/O error to applications

How to inject these faults?

errfs - a FUSE file system to inject file-system faults



Analyze Distributed Storage Services

Behavior of eight distributed systems

Broad spectrum of replication and consensus protocols

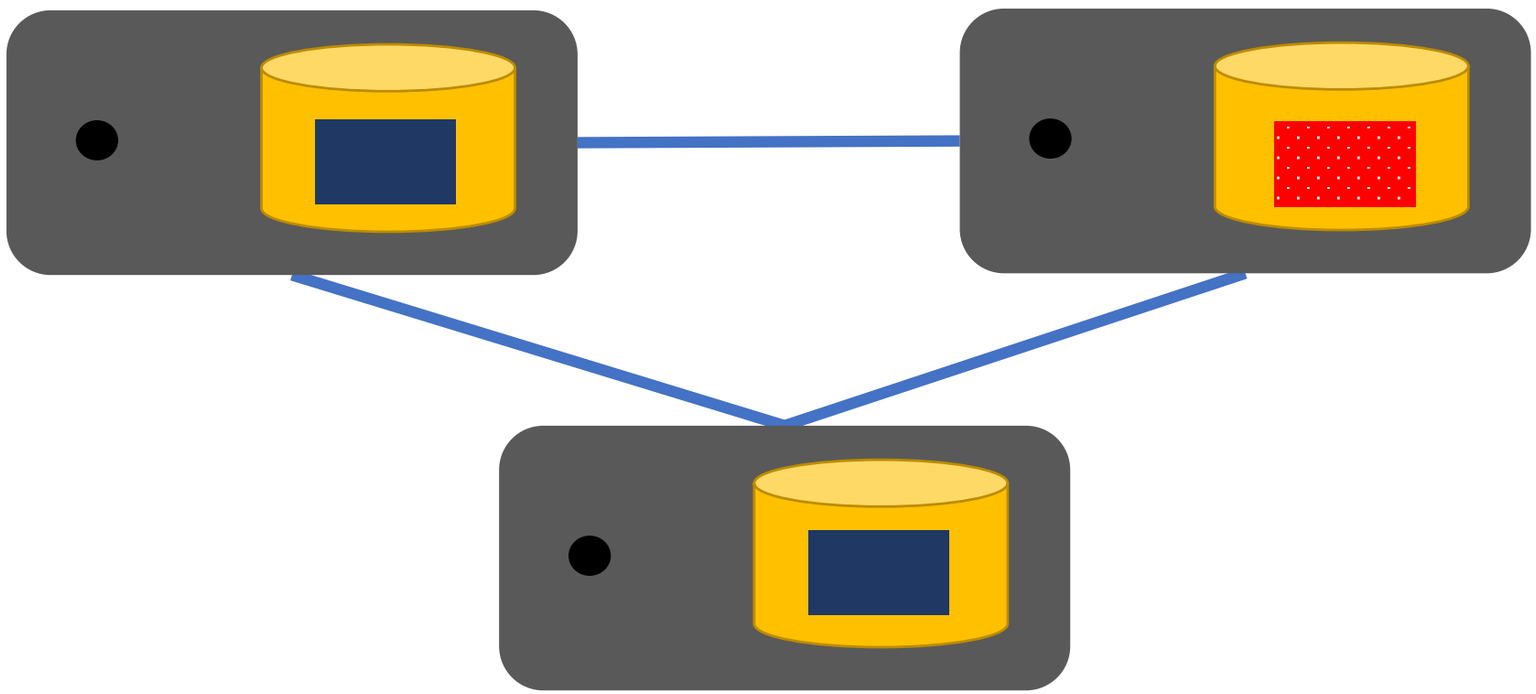
Replicated state machines
- ZooKeeper (uses ZAB for consensus)
- LogCabin, CockroachDB, and RethinkDB (uses RAFT for consensus)

Primary backup replication
- MongoDB
- Redis
- Kafka (in-sync replicas for leader election)

Dynamo-style quorum
- Cassandra (decentralized, no leader/follower)

Common Expectation?

Fault Model: A single fault to one block in only one replica

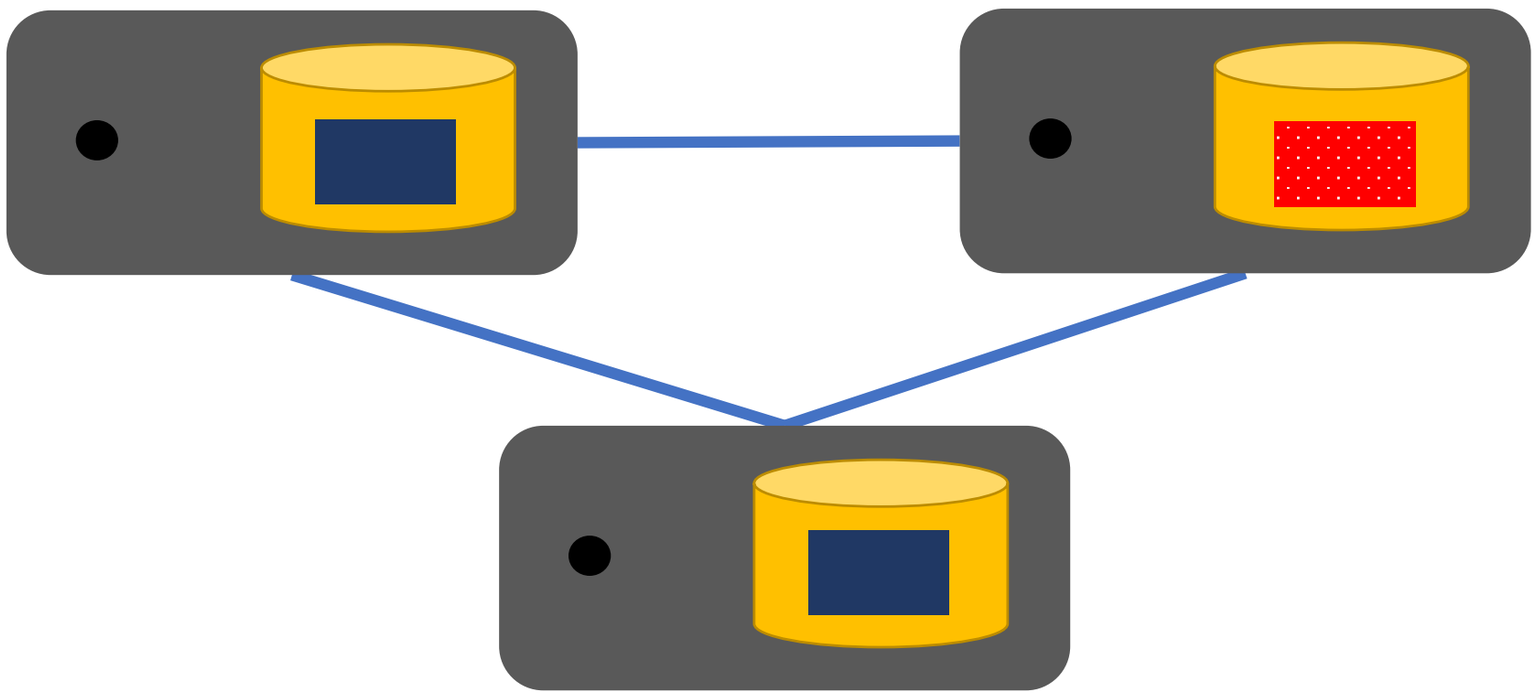


What global properties should hold?

Redundancy would enable recovery from local file-system

Common Expectation?

Fault Model: A single fault to one block in only one replica



Expectations from the softwares after putting the faults

Committed data should not be lost

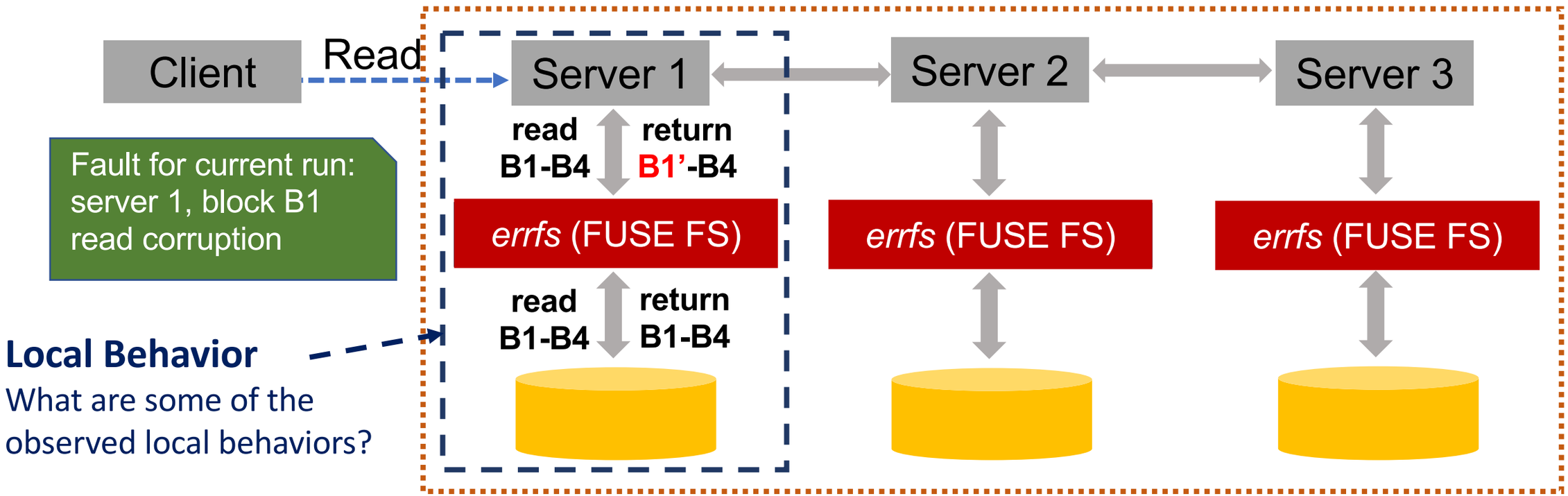
Queries should not silently return corrupted data

Cluster should be available for reads and writes

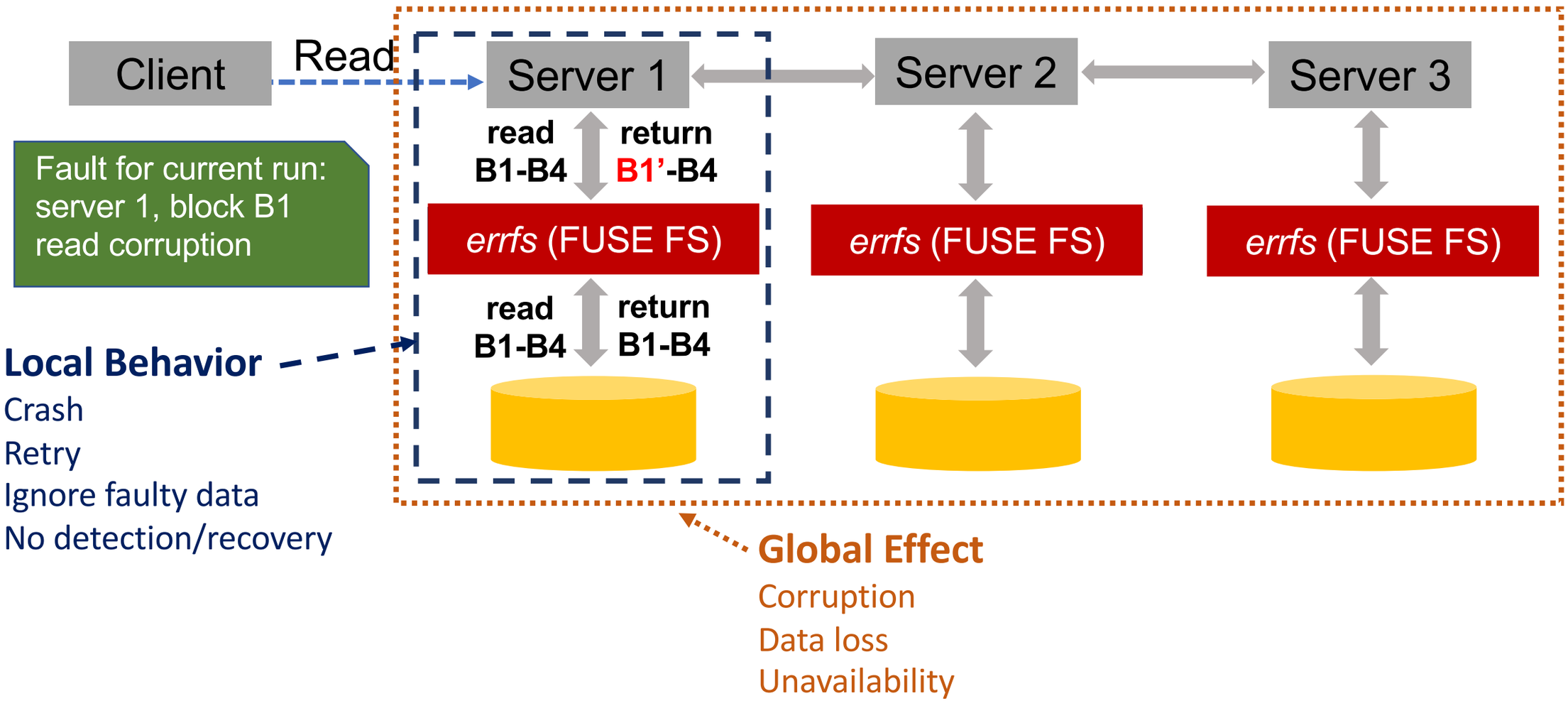
Queries should not fail after retries

Redundancy would enable recovery from local file-system

Difference btwn global and local behavior?



Difference btwn global and local behavior?



Behavior Inference Methodology

Repeat for other blocks, other servers, other faults for different workloads

	Fault	Behavior Observed
Run 1	**Server**: server 1 **Block**: logical data structure X **Fault**: read corruption **Workload**: read	**Local Behavior**: Ignore faulty data **Global Effect**: Data loss
Run 2	**Server**: server 2 **Block**: logical data structure Y **Fault**: write error **Workload**: write	**Local Behavior**: Crash **Global Effect**: None

Contents of Figure 1

Different systems?

Different workloads?

Types of corruption?

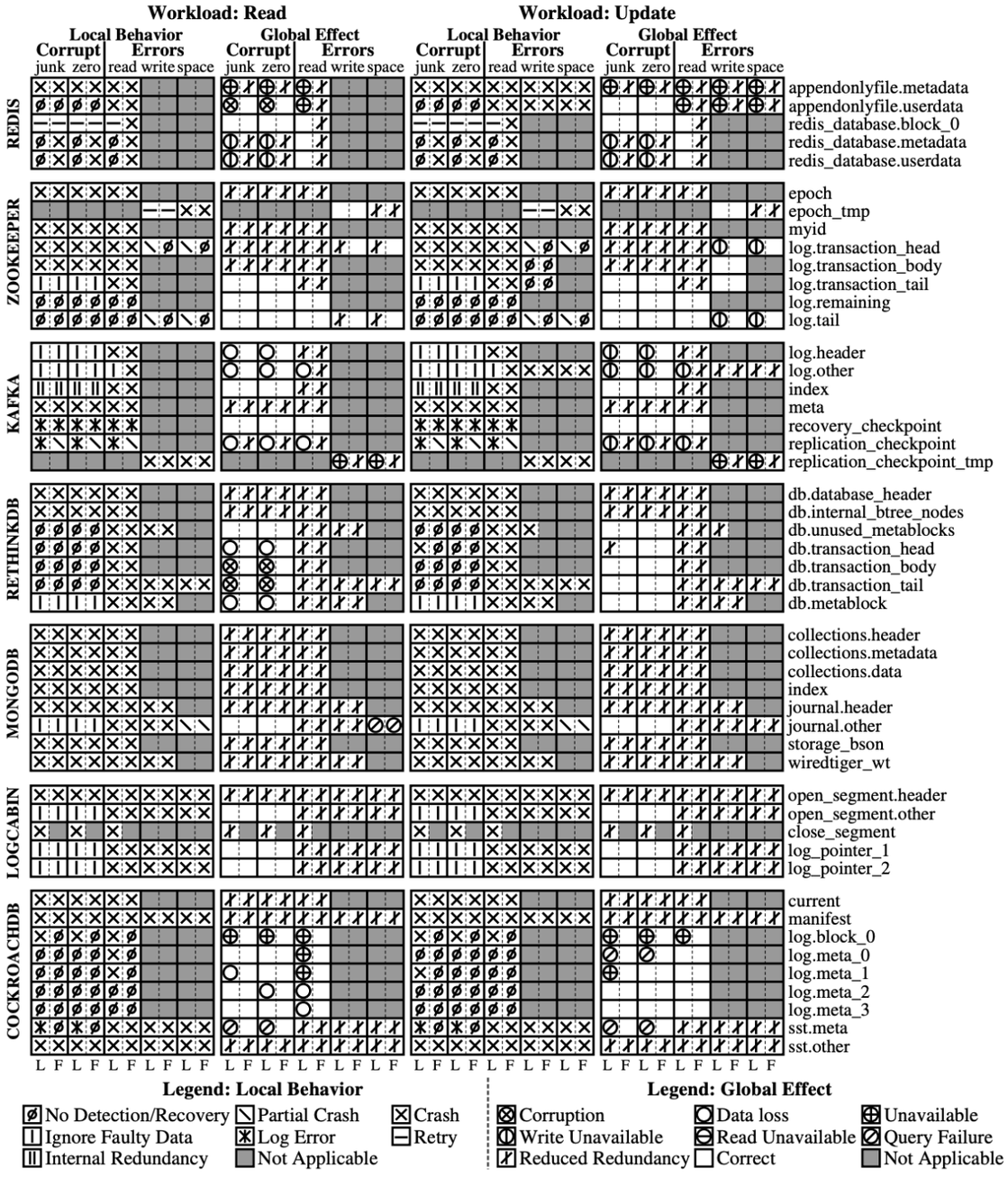
Types of Errors

Local behavior?

Global Effect?

What global effects occur in each system?

Corrupt -> Put junk/zero into one block
read/write/space -> errfs return EIO for read/write and ENOSPC for space.



Redundancy Does Not Imply Fault Tolerance

A single fault in one node can cause catastrophic outcomes

- data loss, corruption, unavailability, and spread of corruption to other intact replicas

	Silent corruption	Unavailability	Data loss	Reduced redundancy	Query failures
Redis	■	■	■	■	■
ZooKeeper		■	■	■	
Cassandra	■	■	■	■	
Kafka		■		■	■
RethinkDB	■			■	
MongoDB				■	
LogCabin				■	
CockroachDB		■	■	■	■

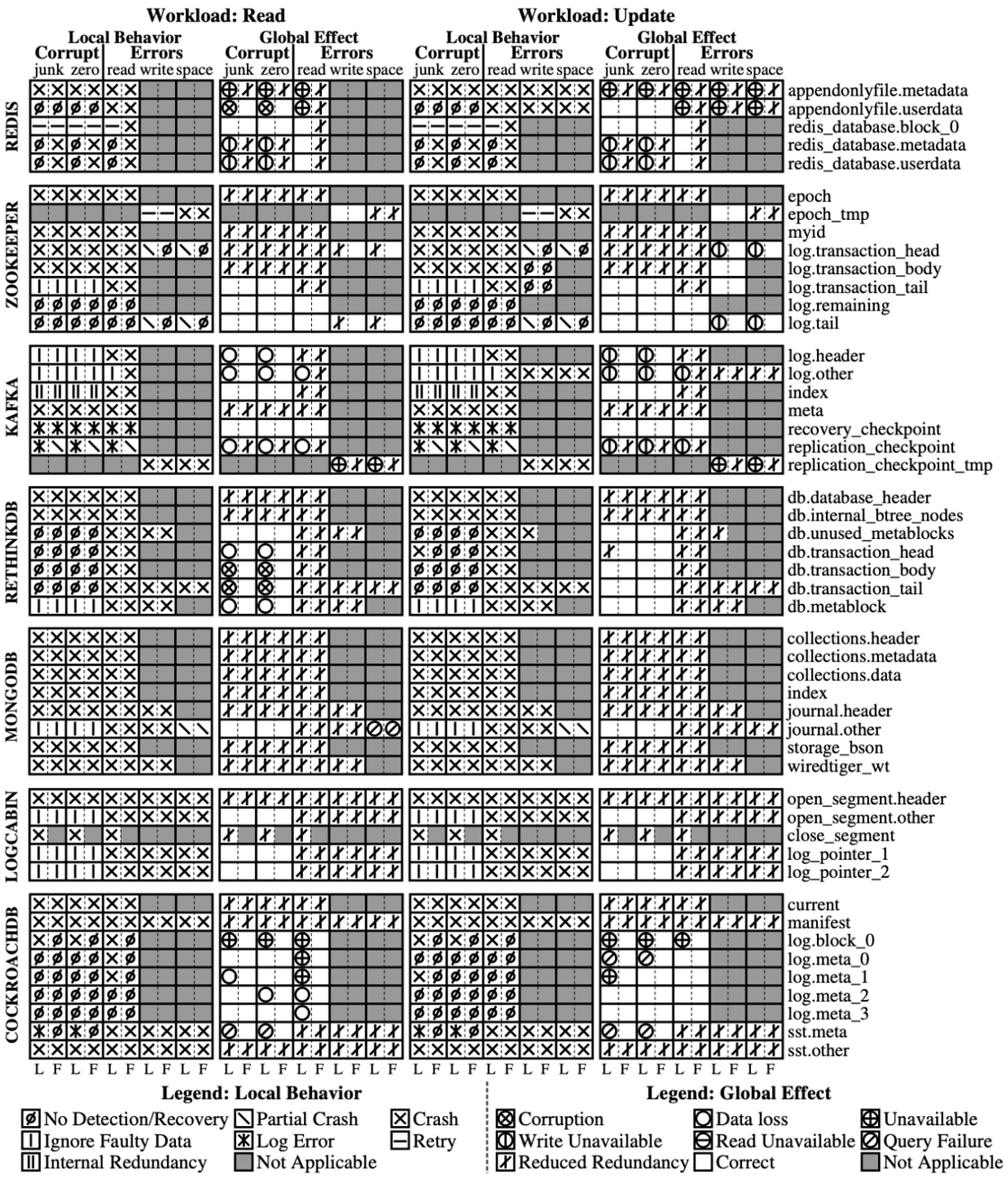
Why does Redundancy Not Imply Fault Tolerance?

- Fundamental problems across multiple systems
  – not just implementation bugs!
- Faults are often undetected locally
  – leads to harmful global effects
- On detection, crashing is common action
  – redundancy underutilized
- Crash and corruption handling are entangled
  – loss of committed data
- Unsafe interaction btwn local + global distributed protocols
  - can spread corruption or data loss

What is most common local reaction?

Crashing leads to reduced redundancy and imminent unavailability

Persistent fault -- Requires manual intervention

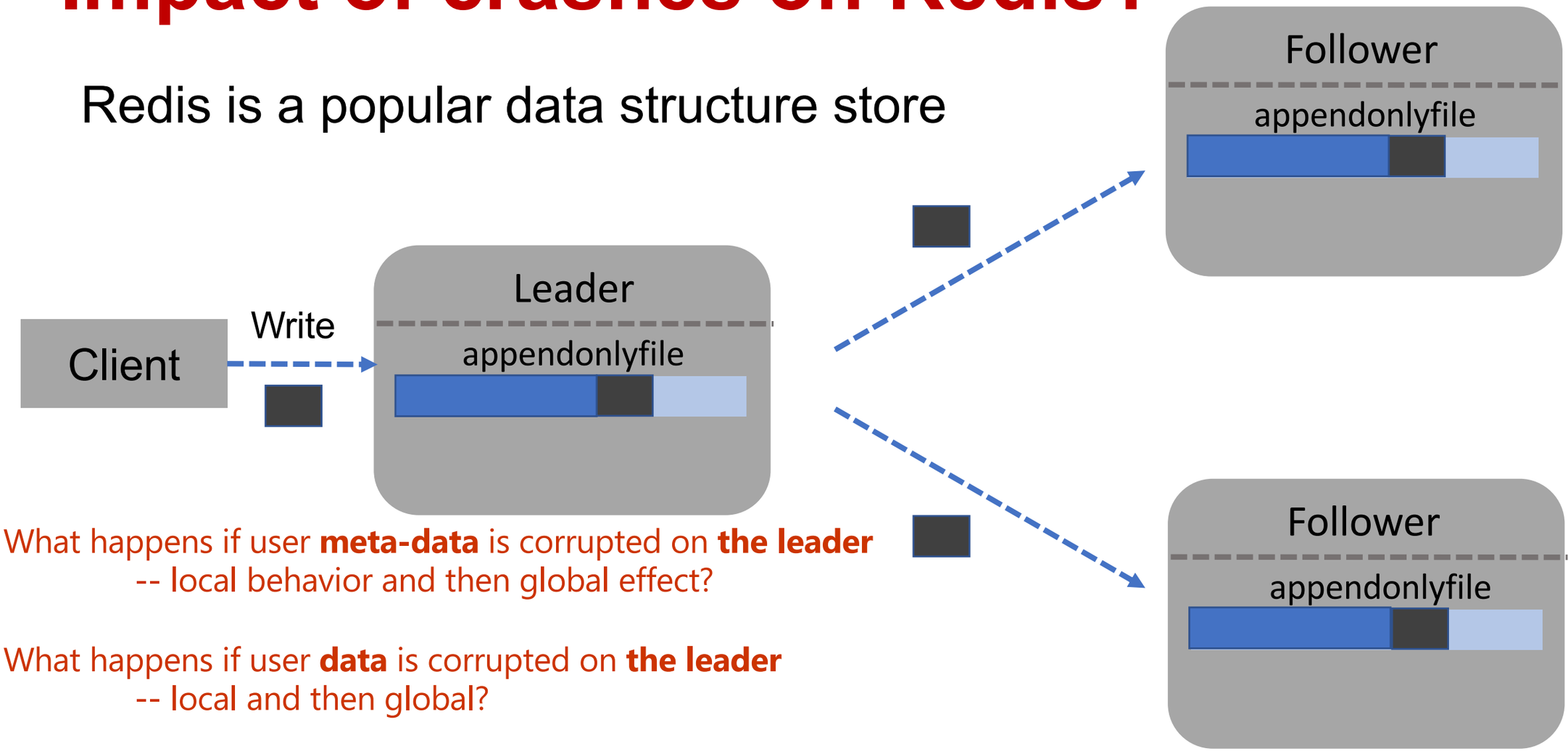


Fundamental Problems: Summary

	Locally undetected faults?	Crashing - common local action?	Crash & corruption handling entangled?	Unsafe interaction of local & global protocols?	Redundancy underutilized for recovery?
Redis	■	■		■	■
ZooKeeper	■	■	■		■
Cassandra	■	■		■	■
Kafka		■	■	■	■
RethinkDB	■	■	■		■
MongoDB		■	■		■
LogCabin		■	■		■
CockroachDB	■	■			■

Impact of crashes on Redis?

Redis is a popular data structure store



What happens if user **meta-data** is corrupted on **the leader**
-- local behavior and then global effect?

What happens if user **data** is corrupted on **the leader**
-- local and then global?

What if this happens on a **follower?**

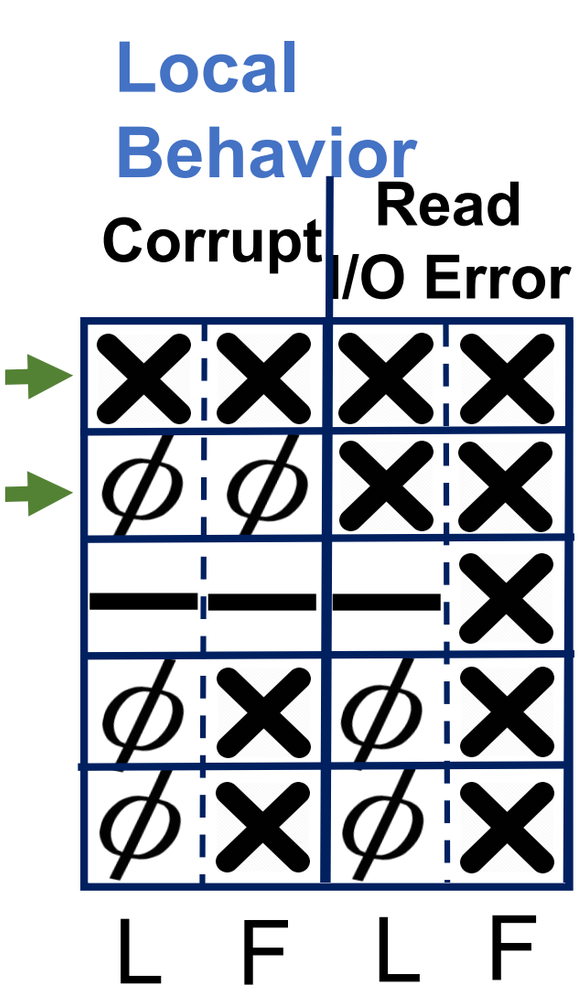
Focus on later reads

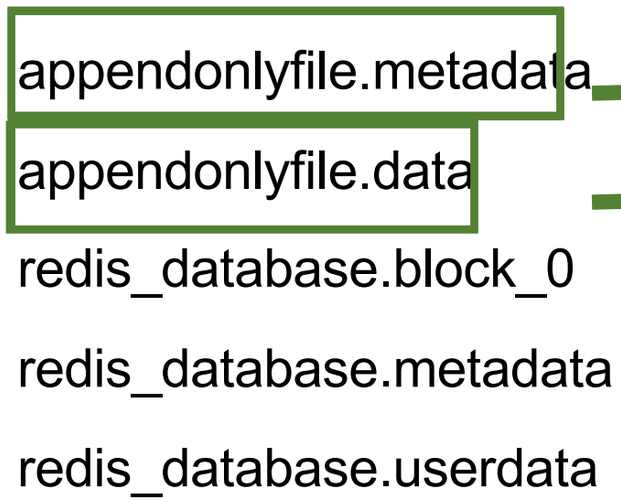
Make sure to contrast the local with the global effect.
For example, Corrupt on LEADER, local => crash while globally, the service becomes unavailable.

What happens if user meta-data is corrupted on the leader
-- local behavior and then global effect?

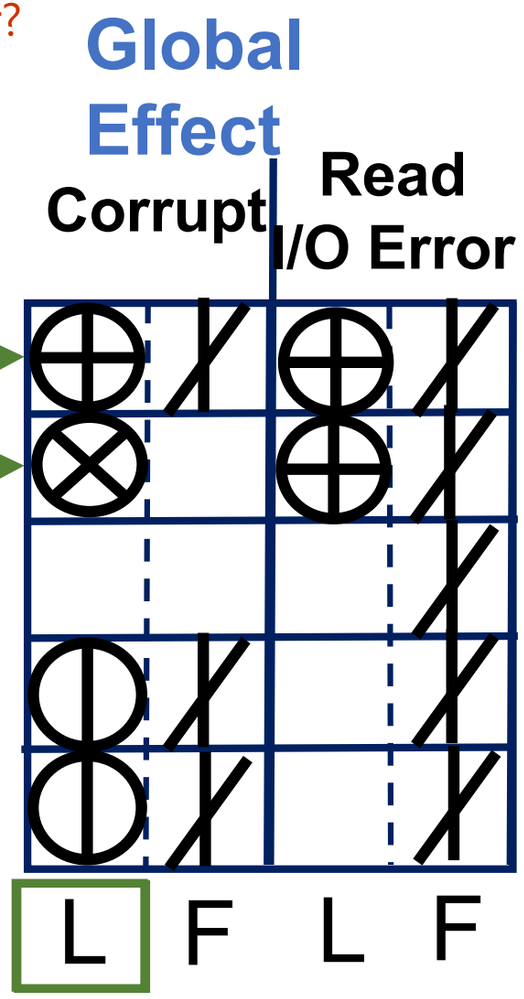
What happens if user data is corrupted on the leader
-- locally and then global?

What if this happens on a follower?





Read Workload



Local Behavior

- Crash
- No Detection/ No Recovery
- Retry

Global Effect

- Unavailability
- Reduced Redundancy
- Corruption
- Correct
- Write Unavailability

What happens if user meta-data is corrupted on the leader
-- locally and then globally?

No checksums to detect corruption
Leader crashes due to failed deserialization
No automatic failover - cluster ***unavailable***

What happens if user data is corrupted on the leader
-- locally and then globally?

No checksums to detect corruption
Leader returns corrupted data on queries
Corruption propagation to followers

What if this happens on a follower?

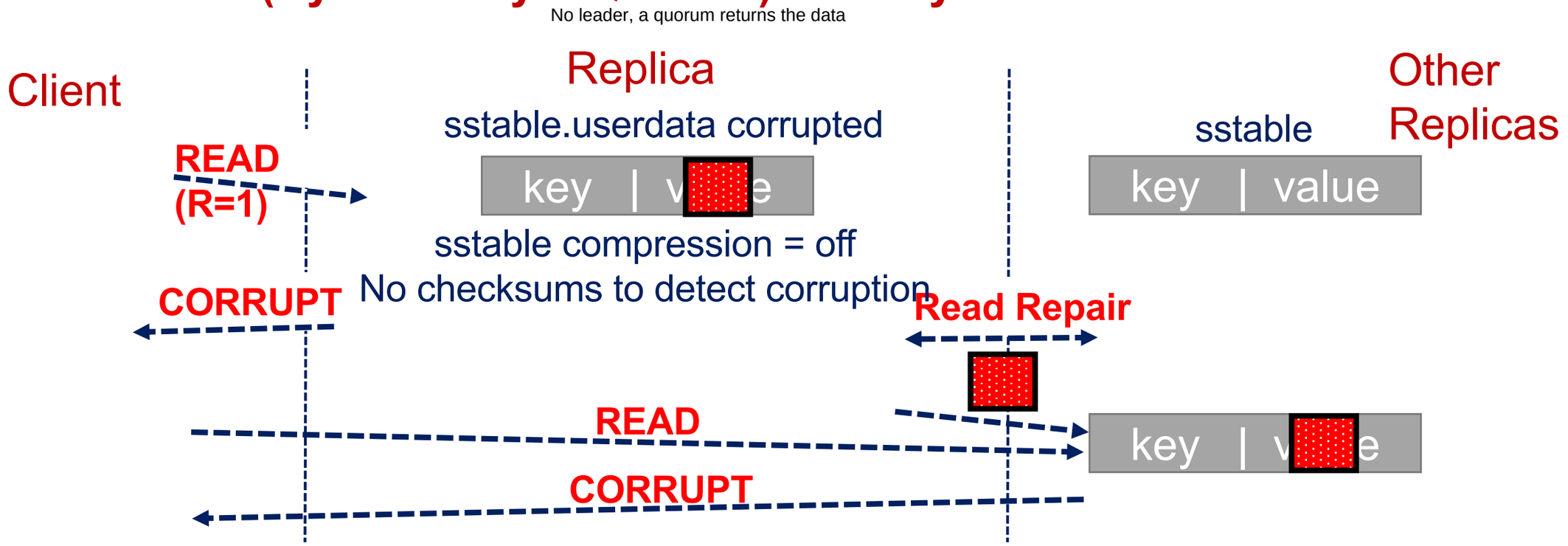
Same local behaviors
Follower crashes → reduced redundancy
Follower has bad data → nothing bad

Impact of Undetected Faults in Cassandra?

Undetected faults may lead to harmful global effect

- Locally undetected corruption → global silent corruption

Cassandra (Dynamo-Style Quorum): Locally Undetected Fault



Need for end-to-end integrity and error handling

What if can’t distinguish between crash and corruption??

Kafka Message Log

Checksums are used to determine if updates have been persisted to local disk

Checksum can be used for:
- Integrity
- Was all the data written? If there is a crash, we can calculate the checksum on data committed and compare with this.

A mismatch means that the data was not fully committed.
The problem is that we do not know which is which when the checksum mismatching.

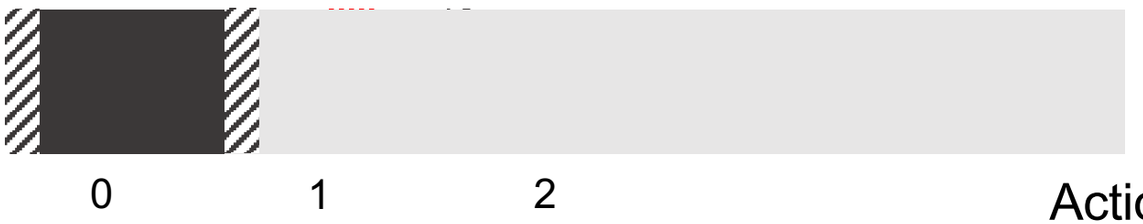
Append(log, entry 2)

Checksum mismatch

Action: Truncate log at 1

Lose uncommitted data -- OK

How can a local node determine if a checksum mismatch is due to corruption or crash?



Disk corruption

Checksum mismatch

Action: Truncate log at 0

Lose committed data!

Need for discerning corruptions due to crashes from other type of corruptions

Unsafe Interaction btwn Local & Global Protocols

Kafka: Message log at Node 1

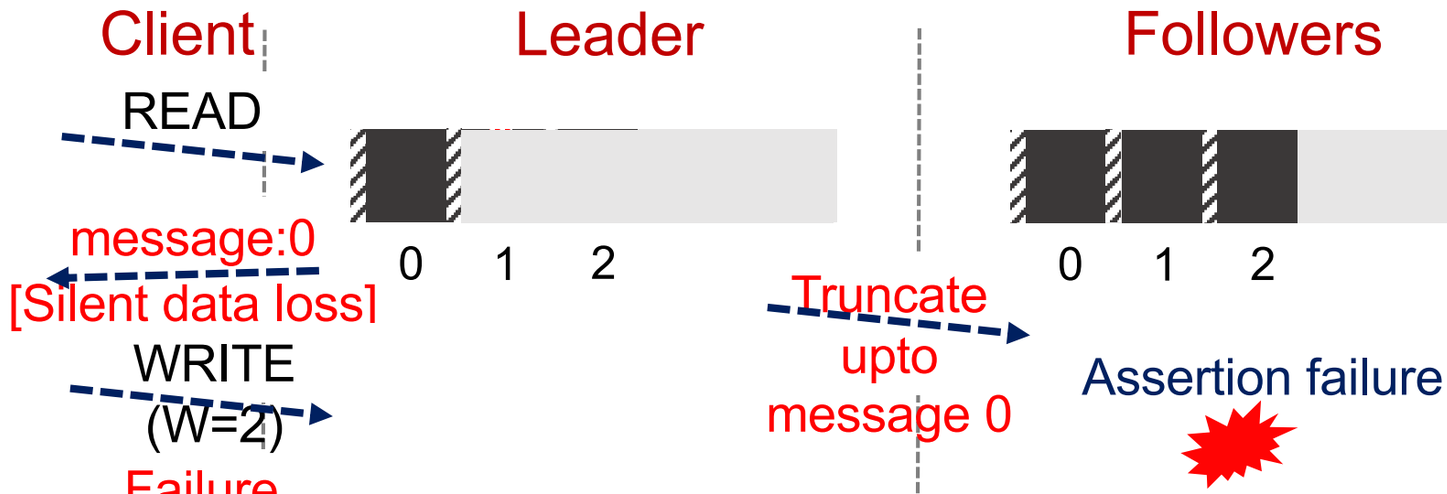
Local Behavior

Disk corruption
Checksum mismatch
Action: Truncate log at 0
Lose committed data!

Set of in-sync replicas

Node1 with truncated log not removed from in-sync replicas

Node 1 elected as leader



Unsafe interaction between local behavior and leader election protocol leads to data loss and write unavailability

Need for synergy between local behavior and global protocol

Conclusion

Most distributed systems not yet resilient

Always detect faults – important in layered stacks on commodity hardware

Detecting faults and not using redundancy to recover is undesirable

Cannot always assume corruption to be caused by a crash

Local behavior has implications for distributed systems

Our study provides directions for more robust distributed storage design

Our fault injection framework available online: http://research.cs.wisc.edu/adsl/Software/cords/